

Statistical Programming Languages

Variables, Vectors, Matrices, etc.

Katarzyna Reluga¹

¹Acknowledgements: Thanks to Manuel Pfeuffer, Maarten Jung, and Tobias Wistub for sharing notes from earlier editions.

Variables

- ▶ Variables store objects
- ▶ Objects can have any type of structure and class
- ▶ We can perform operations on variables

```
a <- 2          # a is numeric
exp(a)
## [1] 7.389056
b <- 3          # b is numeric
a + b
## [1] 5
c <- "hello"    # c is character
c(a, b)         # vector with elements a and b
## [1] 2 3
```

Constructing vectors

- ▶ Data structure equivalent to mathematical vectors
- ▶ Entries must be of the same type (not necessarily numeric)
- ▶ Numerical vectors support appropriate mathematical operations

```
a <- c(1, 2, 3); a           # Basic constructor
## [1] 1 2 3

b <- 1:3; b                 # Construct as range
## [1] 1 2 3

c <- seq(1, 3, by = 1); c   # Construct using seq()
## [1] 1 2 3

d <- rep(1.5, 3); d        # Construct using rep()
## [1] 1.5 1.5 1.5

e <- c("h", "a", "l", "l", "o"); e # character vector
## [1] "h" "a" "l" "l" "o"

f <- c("a", 2:4); f        # character vector
## [1] "a" "2" "3" "4"
```

Random vectors

- ▶ Functions to create random vectors from probability distributions have the form `r*()`, e.g.,
 - ▶ `rnorm()` - normal distribution
 - ▶ `rpois()` - poisson distribution
 - ▶ `rexp()` - exponential distribution
- ▶ When working with random functions you should set a seed first using the function `set.seed()` to make the result reproducible

```
set.seed(1211)
r.norm <- rnorm(3, mean = 0, sd = 1); r.norm
## [1] -0.3219005  0.8411735 -0.8128960
```

Analogously functions `d*()`, `p*()`, `q*()` return the density, distribution and quantile functions.

Accessing vector elements (1)

- ▶ Access elements of a vector through brackets []
- ▶ `a[i]` gives *i*-th element of vector `a`
- ▶ Element indexing starts at 1, not 0
- ▶ A negativ index returns all elements but the indexed ones

```
a <- c(2, 3, 1, 4)
```

```
a[3]
```

```
## [1] 1
```

```
a[1:2]
```

```
## [1] 2 3
```

```
a[-c(1, 3)]
```

```
## [1] 3 4
```

Accessing vector elements (2)

```
a
## [1] 2 3 1 4
a < 3
## [1] TRUE FALSE TRUE FALSE
which(a < 3)
## [1] 1 3
a[a < 3]
## [1] 2 1
a[which(a < 3)]
## [1] 2 1
```

Vector algebra (1)

Vectors support mathematical operations.

```
a <- b <- c(1, 2, 3)
a / 2          # divide by a scalar
## [1] 0.5 1.0 1.5
a * 2          # multiply by a scalar
## [1] 2 4 6
a * b          # element-wise multiplication
## [1] 1 4 9
a %*% b        # inner product: 1 + 4 + 9
##           [,1]
## [1,]      14
```

Vector algebra (2)

```
a; b
## [1] 1 2 3
## [1] 1 2 3
a %% b          # outer product: (i, j) = a[i] * b[j]
##             [,1] [,2] [,3]
## [1,]        1    2    3
## [2,]        2    4    6
## [3,]        3    6    9
t(a)           # transpose (returns a matrix!)
##             [,1] [,2] [,3]
## [1,]        1    2    3
```


Basic functions (1)

```
set.seed(123)
x <- rexp(1000, rate = 2) # rate parm. lambda
length(x)                # get the length of x
## [1] 1000
head(x, n = 2)           # to see the first 2 elements of x
## [1] 0.4217286 0.2883051
mean(x)                  # ?median
## [1] 0.5149896
sd(x)                    # ?var
## [1] 0.5022022
min(x); max(x)           # ?range
## [1] 0.0004129869
## [1] 3.605504
```

Basic functions (2)

```
quantile(x, c(0.25, 0.75))
##           25%           75%
## 0.1533463 0.7133176
sort.x <- sort(x, decreasing = FALSE)
head(sort.x, 2)
## [1] 0.0004129869 0.0006716992
order.x <- order(x, decreasing = FALSE)
head(order.x, 2)
## [1] 439 881
x[c(439, 881)]
## [1] 0.0004129869 0.0006716992
```

Missing values

```
x <- c(1.3, 4, NA, 2)
mean(x)                # NA: Not Available
## [1] NA
mean(x, na.rm = TRUE)
## [1] 2.433333
3 / 0                  # Inf: limit of 3/0
## [1] Inf
log(-2)                # NaN: Not a Number
## Warning in log(-2): NaNs produced
## [1] NaN
sqrt(-5)               # NaN: Not a Number
## Warning in sqrt(-5): NaNs produced
## [1] NaN
sqrt(-5 + 0i)          # Works fine for complex num.
## [1] 0+2.236068i
```

Lists in R

- ▶ Lists contain components, possibly of different types (e.g., numeric values, character vectors, lists, functions, matrices, arrays,...)
- ▶ Components of lists can have names

```
list(1:2, "Hallo") # unnamed el.
## [[1]]
## [1] 1 2
##
## [[2]]
## [1] "Hallo"
list(numeric = 1:2, character = "Hallo") # named el.
## $numeric
## [1] 1 2
##
## $character
## [1] "Hallo"
```

Accessing list elements

- ▶ Access elements of a list through `[[ ]]` or `$` if the element has a name
- ▶ `[ ]` returns a sublist

```
family <- list(father = "Joseph", mother = "Annie",
              n.children = 2)
family$father # Equivalent: family[["father"]]
## [1] "Joseph"
family[[1]]
## [1] "Joseph"
family[2:3]
## $mother
## [1] "Annie"
##
## $n.children
## [1] 2
```

unlist() function

```
a <- unlist(family)
a
##      father      mother n.children
##  "Joseph"    "Annie"         "2"
class(family)
## [1] "list"
length(family)
## [1] 3
class(a)
## [1] "character"
length(a)
## [1] 3
```

Constructing matrices (1)

Matrices are constructed using the `matrix()` function.

```
A <- matrix(1:16, nrow = 4); A
```

```
##      [,1] [,2] [,3] [,4]
```

```
## [1,]    1    5    9   13
```

```
## [2,]    2    6   10   14
```

```
## [3,]    3    7   11   15
```

```
## [4,]    4    8   12   16
```

```
# byrow default FALSE
```

```
B <- matrix(1:16, nrow = 4, byrow = TRUE); B
```

```
##      [,1] [,2] [,3] [,4]
```

```
## [1,]    1    2    3    4
```

```
## [2,]    5    6    7    8
```

```
## [3,]    9   10   11   12
```

```
## [4,]   13   14   15   16
```

Constructing matrices from vectors

Functions `cbind()` and `rbind()` bind vectors by column or row, respectively.

```
# Concatenation
set.seed(123)
x <- rbinom(4, size = 5, prob = 0.6)
y <- rbinom(4, size = 5, prob = 0.1)
rbind(x, y)
##      [,1] [,2] [,3] [,4]
## x      4    2    3    2
## y      2    0    0    1
cbind(x, y)
##      x y
## [1,] 4 2
## [2,] 2 0
## [3,] 3 0
## [4,] 2 1
```


Accessing matrix elements

- ▶ Access elements of a matrix through brackets [,]
- ▶ `A[i, j]` gives element in row `i` and column `j` of matrix `A`
- ▶ Row and column indexing starts at 1, not 0
- ▶ Same principles for rows and columns as for vectors

```
A <- matrix(1:12, nrow = 3, ncol = 4); A
##      [,1] [,2] [,3] [,4]
## [1,]    1    4    7   10
## [2,]    2    5    8   11
## [3,]    3    6    9   12
A[1:2, 3]
## [1] 7 8
A[-1, 1]
## [1] 2 3
A[1, c(1, 4)]
## [1] 1 10
```

Matrix dimensions

- ▶ Various functions deal with the dimensions of a matrix
- ▶ Often useful when looping through rows or columns

```
X <- matrix(1:10, 2, 5); X
##      [,1] [,2] [,3] [,4] [,5]
## [1,]    1    3    5    7    9
## [2,]    2    4    6    8   10
dim(X)
## [1] 2 5
nrow(X)
## [1] 2
ncol(X)
## [1] 5
length(X)
## [1] 10
```

Matrix dimension names

- ▶ Rows and columns can be named
- ▶ Once named, matrix elements are accessible through names

```
B <- matrix(1:4, nrow = 2)
dimnames(B) <- list(c("a", "b"), c("a", "b"))
B["a", "a"]
## [1] 1
colnames(B) <- c("b", "a")
rownames(B) <- c("b", "a")
B
##      b a
## b 1 3
## a 2 4
B["a", "a"]
## [1] 4
```

Constructing matrices from vectors (2)

A vector can be transformed to a matrix by changing its dimension.

```
x
## [1] 4 2 3 2
dim(x) <- c(2, 2)
x
##      [,1] [,2]
## [1,]    4    3
## [2,]    2    2
```

Sums and Products

```
X <- matrix(1:20, 4, 5)
sum(X)
## [1] 210
prod(X)
## [1] 2.432902e+18
colSums(X)
## [1] 10 26 42 58 74
rowSums(X)
## [1] 45 50 55 60
colMeans(X)
## [1] 2.5 6.5 10.5 14.5 18.5
rowMeans(X)
## [1] 9 10 11 12
```

Cumulative functions on vectors and matrices

```
a <- 1:5
a
## [1] 1 2 3 4 5
cumsum(a)
## [1] 1 3 6 10 15
cumprod(a)
## [1] 1 2 6 24 120
###
b <- c(3, 2, 1, 2, 1, 0, 4, 2, -3)
cummin(b)
## [1] 3 2 1 1 1 0 0 0 -3
cummax(b)
## [1] 3 3 3 3 3 3 4 4 4
```

Special matrices (1)

```
A <- matrix(1:4, nrow = 2); A
##      [,1] [,2]
## [1,]    1    3
## [2,]    2    4
diag(A)
## [1] 1 4
diag(1:2)
##      [,1] [,2]
## [1,]    1    0
## [2,]    0    2
diag(2)
##      [,1] [,2]
## [1,]    1    0
## [2,]    0    1
```

Special matrices (2)

```
A
##      [,1] [,2]
## [1,]    1    3
## [2,]    2    4
upper.tri(A)
##      [,1] [,2]
## [1,] FALSE TRUE
## [2,] FALSE FALSE
A[upper.tri(A)] <- 0
A
##      [,1] [,2]
## [1,]    1    0
## [2,]    2    4
```


Replacing values

```
replace(A, A < 2, 3)
```

```
##      [,1] [,2]
```

```
## [1,]    3    3
```

```
## [2,]    2    4
```

```
A
```

```
##      [,1] [,2]
```

```
## [1,]    1    0
```

```
## [2,]    2    4
```

```
A[A < 2] <- 3; A
```

```
##      [,1] [,2]
```

```
## [1,]    3    3
```

```
## [2,]    2    4
```

```
A[1, 1] <- 1; A
```

```
##      [,1] [,2]
```

```
## [1,]    1    3
```

```
## [2,]    2    4
```

```
# A <- replace(A, A < 2, 3)
```

Matrix algebra (1)

Matrices support mathematical operations (also works, if one argument is a vector).

```
A <- B <- matrix(1:4, nrow = 2, byrow = TRUE)
A + B          # Elementwise sum
##           [,1] [,2]
## [1,]         2   4
## [2,]         6   8
A - B          # Elementwise difference
##           [,1] [,2]
## [1,]         0   0
## [2,]         0   0
```

Matrix algebra (2)

```
A * B           # Elementwise multiplication
##           [,1] [,2]
## [1,]        1    4
## [2,]        9   16
A %*% B         # Matrix product
##           [,1] [,2]
## [1,]        7   10
## [2,]       15   22
A / B           # Elementwise division
##           [,1] [,2]
## [1,]        1    1
## [2,]        1    1
```

Matrix Inversion

```
A <- matrix(c(1, 2, 5, 7), nrow = 2, ncol = 2)
A_inv <- solve(A) # inverse of A
A %*% A_inv; A_inv %*% A
##           [,1]      [,2]
## [1,] 1.000000e+00 -5.551115e-17
## [2,] 3.330669e-16  1.000000e+00
##           [,1]      [,2]
## [1,]      1 1.332268e-15
## [2,]      0 1.000000e+00
A^(-1)           # elementwise inverse
##           [,1]      [,2]
## [1,]      1.0 0.2000000
## [2,]      0.5 0.1428571
```

Constructing arrays

- ▶ Arrays can store elements (of the same type) in more than two dimensions
- ▶ Two-dimensional arrays are equivalent to matrices
- ▶ Arrays are constructed using the `array()` function.

```
x <- array(1:6, dim = c(1, 3, 2)); x
## , , 1
##
##      [,1] [,2] [,3]
## [1,]    1    2    3
##
## , , 2
##
##      [,1] [,2] [,3]
## [1,]    4    5    6
```

Constructing arrays from vectors

A vector can be transformed to an array by changing its dimension.

```
x <- 1:6; class(x); length(x); dim(x)
## [1] "integer"
## [1] 6
## NULL
dim(x) <- c(1, 3, 2) # yields the same array as before
class(x); length(x); dim(x)
## [1] "array"
## [1] 6
## [1] 1 3 2
```

Accessing array elements

```
x[1, , ] # first rows of the two matrices
##      [,1] [,2]
## [1,]    1    4
## [2,]    2    5
## [3,]    3    6
x[, 1, ] # first columns of the two matrices
## [1] 1 4
x[, , 1] # first matrix
## [1] 1 2 3
```